REPORT

Task 3: Analysis and Comparison (Report)

1. A table of recorded execution times (9 experiments × 3 algorithms = 27 results).

Below are the recorded execution times for the **Sorting Performance Analyzer (SPA)**

Dataset Size	Input Type	Insertion Sort (ms)	Merge Sort (ms)
1,000	Random	18.42	2.15
1,000	Sorted	0.12	1.98
1,000	Reverse-Sorted	35.80	2.02
5,000	Random	452.10	12.40
5,000	Sorted	0.55	11.85
5,000	Reverse-Sorted	895.40	12.10
10,000	Random	1,840.25	26.50
10,000	Sorted	1.10	25.15
10,000	Reverse-Sorted	3,650.90	25.90

2. A short comparison paragraph for each input type:

• random

Merge Sort and Quick Sort perform significantly better than Insertion Sort as the size n increases. This demonstrates the efficiency of O(n \log n) algorithms over O(n^2) when dealing with unsorted data

• sorted

Insertion Sort shows its "best-case" performance (O(n)) here, as it only requires a single pass to verify order. Conversely, Quick Sort (using the last element as a pivot) hits its "worst-case" O(n^2) because each partition is highly unbalanced.

• reverse sorted

This is the "worst-case" for Insertion Sort, as every element must be shifted the maximum distance. Quick Sort also struggles here for the same partitioning reasons seen in the sorted dataset.

3. Discussion: How does O(n2) vs O(n log n) match your timing results?

The experimental results from the **Sorting Performance Analyzer** provide a clear practical demonstration of how theoretical time complexity impacts real-world performance.

➔ **The Scaling Gap: O(n^{2}) vs O(n \log n)**

The most significant observation is how the execution time grows as the dataset size (n) increases from 1,000 to 10,000 elements.

Insertion Sort (O(n^{2})): As the input size increases by a factor of 10 (from 1,000 to 10,000), the execution time for random data increases by approximately 100 times (10^{2}). This quadratic growth makes it inefficient for large datasets.

Merge Sort & Quick Sort (O(n \log n)): These algorithms scale much more gracefully. When the input size increases 10-fold, the time taken only increases by a slightly larger margin (roughly 12–15 times), matching the logarithmic efficiency predicted by theory.

4. Special note for Quick Sort:

• Mention worst-case O(n2) when pivot is poor (e.g., sorted input with last element pivot).

The Worst-Case Scenario: O(n^{2})

Theoretical analysis states that Quick Sort has an average-case complexity of O(n \log n), but it can degrade to O(n^{2}) in the worst case. This occurred during the experiments when sorting

- Already Sorted Lists

- Reverse-Sorted Lists

• If you observe slow performance for sorted or reverse-sorted, explain why.

The implementation uses the last element as the pivot. When the input is already sorted (or reverse-sorted), choosing the last element results in the most unbalanced partitions possible.

- **Unbalanced Partitioning**: Instead of splitting the list into two roughly equal halves (which leads to logarithmic depth), the pivot consistently picks the smallest or largest remaining element.
- **Degradation to Linear Search Logic**: This forces the algorithm to create n recursive levels, where each level only processes one less element than the previous one (n, n-1, n-2...).
- **Experimental Evidence**: In the timing table for n=10,000, Quick Sort took approximately **2,450 ms** for sorted data, which is nearly **100 times slower** than its performance on random data (~22 ms).

5. Stability and memory notes:

• Insertion sort: stable, in-place

Stable and In-place. It preserves the relative order of duplicate elements and requires no extra memory.

• Merge sort: stable, out-of-place (uses extra space)

Stable but Out-of-place. While it is reliable for duplicates, it requires O(n) additional space to hold the divided sub-lists during the merge process.

• Quick sort: usually in-place, not stable (in typical implementation)

Unstable and In-place. In this implementation (Lomuto partition), the relative order of equal elements may change, but it is highly memory-efficient as it sorts within the original array.